

Autonomous-QA heavy-work distribution to thinker.local

Status: CONFIGURED (config + glue only — no heavy build has been run yet, per operator directive). Last updated: 2026-06-30.

TL;DR

All autonomous-QA heavy work (Android APK + androidTest build, the containerized KVM Android emulator, connectedDebugAndroidTest, and the Go/Android backend) is routed to **thinker.local** (192.168.0.228, x86_64, 16 cores, /dev/kvm present, rootless podman 4.9.3, 667 GB free, no host Android SDK — the lava-android-emulator container bundles its own SDK+emulator).

Chosen topology: **Approach B — run-on-thinker (co-locate the whole stack)**. The Containers submodule still boots the emulator (its pkg/emulator/ containerized.go LOCAL-podman path), but executed *on thinker*. The CONTAINERS_REMOTE_* config in submodules/containers/.env registers thinker as remote host #1 for the service-distribution model (the Go backend), which scripts/distribute-api-remote.sh already implements end-to-end.

thinker reachability (confirmed)

Property	Value
Address	thinker.local → 192.168.0.228, port 22
SSH user	milosvasic
SSH key (authenticates)	~/.ssh/id_ed25519 (only existing key; offered + accepted; loaded in agent)
Arch / cores	x86_64 / 16
/dev/kvm	present — crw-rw----+ (group kvm + ACL grants access)
podman	4.9.3 (rootless)

Property	Value
podman.socket	disabled → a DOCKER_HOST=ssh:// remote-podman path is NOT wired today
Android SDK on host	none — the emulator container bundles SDK+emulator

Probe used (no key contents printed): `ssh -o BatchMode=yes milosvasic@thinker.local 'uname -m; nproc; podman --version; ls -l /dev/kvm'`

Why Approach B, not Approach A

Approach A — Containers remote-distribution (pkg/distribution over SSH)

The submodule's `pkg/distribution + CONTAINERS_REMOTE_*` genuinely place *service containers* on a remote host via SSH (`docker/podman run -d + tunnels + health check`). That is the right model for long-running services — and it is exactly what `scripts/distribute-api-remote.sh` already does for the Go backend.

But it does NOT fit the QA *flow*, because:

1. **pkg/emulator/containerized.go has no SSH/remote path** (verified: zero `ssh/remote/DOCKER_HOST` references). `cmd/emulator-matrix --runner= containerized` boots the emulator on the **local** podman only. So the distributor could start a generic emulator container on *thinker*, but the Lava emulator lifecycle glue (`lib-emulator.sh: podman cp` of the baked `adbkey`, `adb connect 127.0.0.1:<port>`, `boot-wait`) would still target *local* podman and *local* loopback — wrong host.
2. **gradle + adb are not relocated by pkg/distribution.** `connectedDebug-AndroidTest` runs on the orchestrating host; its `adb` must reach the emulator. With the emulator on *thinker* and `gradle` here, `adb` would have to cross the LAN over an SSH tunnel to the container's socat bridge — the exact `adb-over-WAN` fragility the operator flagged.
3. **podman.socket is disabled on thinker**, so even the `DOCKER_HOST=ssh:// remote-podman` shortcut is unavailable without first enabling the user socket.

Net: Approach A splits the tightly-coupled trio (gradle ↔ adb ↔ emulator loopback; on-emulator client ↔ backend) across two hosts. That breaks the onboard → search → download flow.

Approach B — run-on-thinker (chosen)

Sync the repo (+ .env) to thinker and run the orchestrator **on thinker** over SSH. Then build, emulator, adb, connectedAndroidTest, and backend ALL share thinker's loopback:

- gradle's adb and the emulator are on the same host → no tunnels.
- the on-emulator client reaches the Go backend via `adb reverse tcp:8443` (same host) or the on-device api-app via on-device loopback → no WAN.
- `containerized.go` runs its supported LOCAL-podman + `/dev/kvm` path — exactly what thinker provides. The emulator is still "container-managed by the Containers submodule", just executed on thinker.

This is the only topology that makes the full onboard → search → download QA flow work, so it is the recommendation.

Per-workload routing

Workload	Where it runs	Mechanism
Go backend (lava-api-go)	thinker	Already covered by scripts/distribute-api-re (build image → scp → thinker-up.sh → health). For located QA flow, lib-backend.sh's ./start.sh runs <i>thinker</i> via remote_run.
Emulator image build	thinker	podman build of the lava-android-emulator Container thinker (or pull ghcr.io/vasic-digital/lava-android-emulator:api34-x86_64).
Containerized KVM emulator	thinker	lib-emulator.sh podman run --device /dev/kvm -- thinker (the submodule's containerized.go path)
Gradle APK + androidTest build	thinker	./ gradlew :app:assembleDebug :app:assembleDebug on thinker.
connectedDebugAndroidTest (Challenge70)	thinker	run-iteration.sh on thinker; ANDROID_SERIAL = the emulator serial.

Workload	Where it runs	Mechanism
Vision analysis / evidence aggregation	here or thinker	remote_fetch the curated .lava-ci-evidence/aut <date>/ tree back; raw media stays on thinker.

Exact commands

Prereqs are config-only and already in place: submodules/containers/.env (CONTAINERS_REMOTE_*, gitignored) and scripts/autonomous-qa/lib-remote.sh.

```
# 0. (config already done) confirm thinker fit – reachability +
    podman + kvm
source scripts/autonomous-qa/lib-remote.sh
remote_preflight

# 1. mirror the repo onto thinker (includes the gitignored .env;
    excludes
#    build/.git/recordings). Faithful mirror via rsync --delete.
remote_sync_repo

# 2. run the FULL matrix on thinker (build + emulator + gradle +
    backend all
#    co-located on thinker's KVM). One backend at a time (operator
    rule).
remote_run "ANDROID_HOME=\$HOME/Android/Sdk scripts/autonomous-qa/
    run-matrix.sh \
    --backend goapi --subsets rutracker --queries 1080p"

# 3. pull curated evidence back (raw media stays on thinker)
remote_fetch ".lava-ci-evidence/autonomous-qa/$(date +%F)/goapi/
    summary.md" \
    ".lava-ci-evidence/autonomous-qa/$(date +%F)/goapi/summary.md"
```

The Go backend can alternatively be distributed independently with the existing path (no repo sync needed — it ships an image tarball):

```
./scripts/distribute-api-remote.sh thinker.local          # boot Go
    API on thinker
./scripts/distribute-api-remote.sh --tear-down thinker.local
```

Change-list — what the orchestrator scripts need to run heavy work on thinker

The current orchestrator (`run-matrix.sh`, `run-iteration.sh`, `lib-emulator.sh`, `lib-backend.sh`) is fully **local-host** today. Two ways to land Approach B:

Option 1 (no edits to the existing scripts — preferred first step): drive them unchanged *on thinker* via `lib-remote.sh`:

- `remote_sync_repo` then `remote_run "scripts/autonomous-qa/run-matrix.sh ..."`.
- The scripts then see thinker as their local host: local podman (`/dev/kvm`), local `$ANDROID_HOME/platform-tools/adb` (provided by the emulator container / a thinker SDK), local `./gradlew`, local `curl 127.0.0.1:8443`. No script edit required because "local" now means thinker.
- Caveat to resolve on thinker before the first run: `ANDROID_HOME / adb` must exist on thinker for the host-side adb calls in `lib-emulator.sh / lib-backend.sh / run-iteration.sh`. The emulator container bundles the SDK *inside* the container; the host-side adb client still needs platform-tools on thinker (install user-local platform-tools under `~/Android/Sdk`, no sudo) OR switch those adb calls to `podman exec <emu-container> adb ....`

Option 2 (explicit thinker awareness inside the scripts): add an opt-in `--remote thinker` flag that wraps each heavy step in `remote_run`:

- `run-matrix.sh`: source `lib-remote.sh`; when `--remote` set, `remote_sync_repo` once, then re-exec itself on thinker (`remote_run "run-matrix.sh <same args>"`) and `remote_fetch` the summary. Everything below stays unchanged (it runs on thinker).
- `lib-emulator.sh`: no change for Option 2 (it already uses local podman, which is thinker's podman when the script runs on thinker). For a *split* topology it would need `podman --url ssh://... or DOCKER_HOST=ssh://... + adb tunnels` — NOT recommended (see Approach A rejection); requires enabling `podman.socket` on thinker first (`systemctl --user enable --now podman.socket`, no sudo).
- `lib-backend.sh`: no change for Option 2 (the `./start.sh` Go-API bring-up and `curl 127.0.0.1:8443` run on thinker). For the *split* topology, point the client at `https://thinker.local:8443` and drop `adb reverse` — but co-located is simpler and is what Approach B does.

- `run-iteration.sh`: no change for Option 2 (`gradle + adb + screenrecord` all run on thinker where the emulator is).

Recommended sequence: ship `lib-remote.sh` (done) → use **Option 1** for the first real run (zero edits, lowest risk) → graduate to Option 2's `--remote` flag only if a one-command local entry point is wanted. The only thinker-side prerequisite is a host `adb` (platform-tools under `~/Android/Sdk`) OR rerouting the host-side `adb` calls through `podman exec`.

Does `scripts/distribute-api-remote.sh` already cover the Go backend on thinker?

Yes — fully. It: loads `.env` (`LAVA_API_GO_REMOTE_HOST` default `thinker.local`, `LAVA_REMOTE_HOST_USER`), verifies SSH + `podman` on the remote, builds the `lava-api-go` OCI image if absent, `scp`'s the image tarball + `deployment/thinker/{thinker.local.env, thinker-up.sh}` + TLS certs (merging the operator `.env`'s `LAVA_AUTH_*/transport` block at distribute time, never committed), runs `thinker-up.sh` on thinker, and verifies `https://thinker.local:8443/health`. It also has a `--tear-down` inverse. The autonomous-QA flow reuses this for the Go backend; only the *emulator* + *gradle* half is new (Approach B / `lib-remote.sh`).

Constitutional notes

- §6.H: submodules/containers/`.env` and the operator `.env` are gitignored; no secret is committed or echoed; only an SSH key *path* is stored.
- §6.U: no `sudo/su` anywhere; thinker-side installs are user-local (`~/Android/Sdk`, `systemctl --user`).
- §6.X/§6.AG/§6.AH: the emulator runs INSIDE a `podman` container on thinker's KVM — never host-direct, never a live device.
- §6.W: thinker is a LAN host, not a git remote — unaffected by the GitHub/GitLab- only mirror rule.